

GATEWEB · EXAM TERMS

考前速讀：說明 + 專有名詞

關網資訊筆試四大範圍：一段一段讀，名詞直接嵌在說明裡

30 題全選擇、30 分鐘，不考數學、不寫程式

AI Agent 與工具呼叫

這部分在講的是：怎麼讓語言模型不只會聊天，而是真的能把事情做完。

我們用的**大型語言模型 (LLM, Large Language Model)**，本質上是根據前面的字去預測下一個字，所以它的輸出是機率性的。文字被切成一個個**token** 來處理，計費和長度上限都用 token 算，一次能處理多少 token 叫**上下文視窗 (Context Window)**。因為是機率性的，它有時會很有自信地講錯，這叫**幻覺 (Hallucination)**。你給它的指令叫 **Prompt**，而 **Temperature (溫度)** 控制輸出的隨機程度，數字越低越穩定，所以要它呼叫工具或抽取資料時都用低溫。

模型本身不能上網、不能查資料庫，所以我們給它一組工具，這個機制叫**工具呼叫 (Tool Calling)**，也叫 **Function Calling**)。工具要用 **JSON Schema** 定義名稱、用途和參數，描述寫得清楚模型才選得對。流程是：模型回答「我要用這個工具、參數是這些」，**真正去執行的是你的程式**，程式跑完再把結果交回去。這句話是考試最愛考的重點：模型只提出，不執行。

如果讓模型自己判斷「資料不夠，我再查一次」「夠了，可以回答」，一直循環到完成，就叫 **Agent**。它跟**工作流 (Workflow)** 的差別在於：工作流的步驟是程式寫死的，Agent 的步驟是模型當下決定的。最常見的運作模式叫 **ReAct (Reasoning + Acting)**，也就是思考、行動、觀察結果，再重複。為了做得好，Agent 需要**規劃 (Planning)**，把大任務拆成小步驟，也常用 **Reflection** 讓模型檢查自己的結果；還需要**記憶 (Memory)**，短期記憶是這次對話的內容，長期記憶是存進資料庫、需要時再檢索出來。多個 Agent 分工合作叫 **Multi-Agent**，好處是專業分工，代價是慢、貴、難除錯。

當工具和應用越來越多，每個應用都要自己接一次工具很麻煩，所以有了 **MCP (Model Context Protocol)**，由 Anthropic 在 2024 年提出的開放協定，可以想成工具的標準插座，把「N 個應用乘以 M 個工具」的接法變成「N 加 M」。它的訊息格式是 **JSON-RPC 2.0**，MCP Server 提供三種能力：**Tools (可執行的工具)**、**Resources (可讀的資料)**、**Prompts (預先寫好的提示範本)**；傳輸方式本機用 **stdio**，遠端用 **Streamable HTTP**。要注意 MCP 和 Function Calling 不是二選一：前者是工具怎麼接進來的標準，後者是模型本身的能力。

流程一旦變複雜、有分支、要中途停下來等人確認，就會用 **LangGraph** 這類框架管理。它的核心概念是 **State (共享狀態)**、**Node (每個步驟)**、**Edge (下一步去哪)**、**Conditional Edge (依條件分支)**、**Checkpoint (保存狀態，可以中斷、恢復、等人工確認)**。這種「重要動作讓人確認」的設計叫 **Human-in-the-loop**。

最後是**護欄 (Guardrails)**：因為步驟是模型決定的，萬一它一直重複呼叫工具就會燒錢或失控，所以要限制**最大迭代次數**、設定**逾時**、只給**工具白名單**、寫入或刪除等高風險動作要人工確認。每一步的決策軌跡都要記錄下來，這叫**可觀測性 (Observability)**，常見工具是 **LangSmith** 和 **Langfuse**。

RAG 檢索增強生成

這部分在講的是：模型不知道你公司的東西，該怎麼辦。

模型學的是公開資料，不知道你們的作業手冊，也不知道上個月剛公告的規定。重新訓練太貴太慢，所以實務做法是：先去公司資料裡找出相關段落，連同問題一起交給模型，要它根據這些內容回答。這就是 **RAG** (Retrieval-Augmented Generation，檢索增強生成)，它能處理私有資料、最新資料，也能降低幻覺，而且不需要重新訓練模型。

要做到這件事，得先把文件準備好，這段叫離線流程：**文件解析** → **清洗** → **切塊** → **轉成向量** → **存進資料庫**。文件太長不能整份塞進去，所以要**切塊 (Chunking)**；切太大會混進不相關的內容、重點被稀釋，切太小又會斷掉語意，所以段落之間通常保留一點**重疊 (Overlap)**。每一段再用模型轉成一串數字，叫 **Embedding (嵌入向量)**，它的特性是意思接近的內容，向量也會接近；比較接近程度最常用**餘弦相似度 (Cosine Similarity)**，看的是兩個向量的夾角。這裡有個常考的坑：**查詢和文件必須用同一個 embedding 模型**，換模型就要重建整個索引。

這些向量存在**向量資料庫 (Vector Database)**裡，常見的有 **Qdrant**、**Chroma**、**Milvus**，還有 **pgvector**，它是 PostgreSQL 的擴充套件。資料量大時不可能每筆都比對，所以用**近似最近鄰 (ANN, Approximate Nearest Neighbor)**，犧牲一點準確度換速度，常見索引結構有 **HNSW** (圖狀結構，快但吃記憶體) 和 **IVF** (先分群再找)。

查詢時的線上流程是：問題轉成向量 → 找出最相近的前幾筆 (**top-k**) → 重新排序 → 組進 prompt → 生成答案並附上出處。只靠語意找有個問題：統一編號、法條編號這種東西要的是一字不差，所以會同時用關鍵字檢索 **BM25**，兩種合起來叫 **Hybrid Search (混合檢索)**。找回來的候選還可以再用比較精準但比較慢的 **Rerank (重排序，通常用 Cross-encoder 模型)** 挑一次，因為它慢，所以只對 top-k 做。

權限也要注意：不同使用者能看的文件不同，正確做法是用 **Metadata 過濾**，在檢索階段就把沒權限的文件排除，**不能靠 prompt 叫模型不要說**。

最後是評估，分兩段看。檢索端看有沒有找到對的資料，指標是 **Recall@k**、**Hit Rate**、**MRR**；生成端看有沒有照資料回答，指標是 **Faithfulness (忠實度)** 和答案相關性。所以答錯時第一件事是分辨是檢索沒撈到，還是撈到了但生成錯，兩者修法完全不同。另外要分清楚 **RAG** 和**微調 (Fine-tuning，例如 LoRA)**：知識常變動用 **RAG**，要改語氣、格式、特定任務行為才用微調。

前後端調用

這部分在講的是：兩個系統之間怎麼講話。

前端和後端、你的系統和外部服務，溝通都走 **HTTP**：送出一個**請求 (Request)**，收到一個**回應 (Response)**。請求用方法 (**Method**) 表示你想做什麼：**GET** 拿資料、**POST** 新增、**PUT** 整筆取代、**PATCH** 改一部分、**DELETE** 刪除。這裡有個概念叫**冪等 (Idempotent)**，意思是同一個請求做一次和做很多次結果一樣，**GET**、**PUT**、**DELETE** 是冪等的，**POST** 不是。所以會改資料的操作在重試時要加上**冪等鍵 (Idempotency Key)**，確保同一筆訂單不會開出兩張發票。

回應會帶**狀態碼 (Status Code)**：2 開頭是成功，其中 **200** 成功、**201** 已建立、**204** 成功但沒有內容；3 開頭是轉址；4 開頭代表你這邊有問題，**400** 格式錯、**401** 沒有認證 (沒登入或憑證無效)、**403** 有認證但沒權限、**404** 找不到、**409** 衝突、**422** 參數驗證失敗、**429** 請求太多被限流；5 開頭是伺服器出事，**500** 伺服器錯誤、**502** 閘道收到錯誤回應、**503** 服務暫時無法使用、**504** 閘道逾時。**401** 和 **403** 的差別是最常考的一題。

API 設計通常遵循 **REST** 風格：網址用名詞代表資源，動作交給 HTTP 方法，而且是**無狀態 (Stateless)** 的，每個請求都要自己帶齊身分資訊。身分驗證有兩種常見做法：**Session** 是登入狀態存在伺服器，瀏覽器只拿一個 id 放在 **Cookie**；**JWT (JSON Web Token)** 則是伺服器發一張簽過名的通行證，結構是 **Header.Payload.Signature**。要記住 JWT 的 Payload 只是 **Base64URL** 編碼，沒有加密，任何人拿到都看得懂，所以不能放機密；簽章只能防止被竄改，而且發出去很難提前作廢，所以通常設短效期再搭配 refresh token。帶 token 的標準寫法是在標頭放 **Authorization: Bearer <token>**。

還有兩個常考的細節。**CORS (跨來源資源共享)** 源自瀏覽器的**同源政策 (Same-Origin Policy)**：協定、網域、連接埠任一不同就算跨域，伺服器要回應 **Access-Control-Allow-Origin** 才會放行，複雜的請求還會先送一個 **OPTIONS 預檢 (Preflight)**。重點是 **CORS** 是瀏覽器的限制，不是伺服器的安全防護，用程式直接呼叫完全不受影響。另外，AI 應用常要把答案一個字一個字吐出來，這種伺服器單向推送用 **SSE (Server-Sent Events)**，走的是 HTTP；需要雙向即時互動才用 **WebSocket**。

最後是後端效能與穩定性。**非同步 (async)** 的原理是等待 I/O 時把執行權讓出去，讓一個程序同時服務很多請求，但只要中間有一個會卡住的**阻塞 (blocking)** 呼叫，整個**事件迴圈 (Event Loop)** 就會停住。資料庫存取要用**連線池 (Connection Pool)**，預先建立一批連線重複使用，避免每次重建或全部擠在同一條連線。呼叫外部服務遇到逾時或 429 時要重試，而且要用**指數退避 (Exponential Backoff)**，每次重試等待時間越拉越長；服務端則用**限流 (Rate Limit)** 保護自己，避免被打爆或成本失控。

測試與安全

這部分在講的是：怎麼確認東西是對的，以及怎麼不被攻擊。

測試依範圍分成三層，合稱**測試金字塔**：**單元測試 (Unit Test)** 只測一個函式，數量最多、跑最快；**整合測試 (Integration Test)** 把幾個模組接起來測；**端對端測試 (E2E)** 模擬使用者從頭點到尾，最慢所以數量最少。測試時如果會呼叫外部服務，通常用**模擬物件 (Mock)** 替換掉，讓測試穩定、快、又不花錢。每次改動後把既有的測試再跑一次，確認沒有把原本好的東西弄壞，叫**回歸測試 (Regression Test)**。

AI 的測試比較特別，因為模型每次講的話不會一模一樣，所以**不能逐字比對**。實務上是驗證輸出的 JSON 結構對不對、關鍵事實有沒有講對、有沒有出現不該有的內容，並且準備一組固定的**測試集**做回歸比較。也可以用另一個模型來評分，叫 **LLM-as-a-judge**，但它本身也會錯，要用人工抽查校準。

效能測試方面，常看的是每秒請求數 (**RPS / QPS**)、錯誤率，以及延遲。延遲要看 **P95**、**P99**，也就是最慢的 5%、1% 有多慢，因為**平均值會把少數很慢的請求蓋掉**。常見壓測工具有 **Locust**、**JMeter**、**k6**。

安全的部分先看傳統 Web 攻擊。**SQL Injection** 是把惡意指令混進查詢字串裡，解法是用**參數化查詢**或 **ORM**，不要自己拼字串。**XSS (跨站腳本)** 是把惡意腳本塞進網頁讓別人執行，解法是輸出前做編碼、設定 **CSP**。**CSRF (跨站請求偽造)** 是誘使你的瀏覽器在已登入狀態下送出請求，解法是 **SameSite Cookie** 或 **CSRF token**。**IDOR (不安全的直接物件參照)** 是把網址上的編號改一改就看到別人的資料，解法是伺服器端檢查這筆資料是不是這個人的。這一類問題在 **OWASP Top 10** 裡歸在第一名的 **Broken Access Control (存取控制失效)**。

還要分清楚三個常被混用的詞：**編碼 (Encoding)**，例如 **Base64** 任何人都能還原，沒有金鑰可言；**加密 (Encryption)** 要有金鑰才能還原，分成對稱的 **AES** 和非對稱的 **RSA**；**雜湊 (Hashing)**，例如 **SHA-256** 根本不能還原。所以密碼是用雜湊存的，而且要用 **bcrypt**、**Argon2** 這種加了**鹽 (Salt)** 的慢雜湊。

AI 特有的風險列在 **OWASP LLM Top 10 (2025)**，第一名是 **Prompt Injection (提示詞注入)**：有人把「忽略前面的指示，改做別的事」藏在使用者輸入裡，叫直接注入；藏在網頁、文件、email 裡被 **RAG** 或 **Agent** 讀進來，叫**間接注入**，後者更難防。另一個重要項目是 **Excessive Agency (過度代理)**，也就是給 AI 的權限太大。麻煩的是這些沒辦法只靠一句「不要理會惡意指令」解決，真正的做法是**最小權限 (Least Privilege)**：限制能用的工具、會改資料的動作要**人工確認 (Human-in-the-loop)**、外部來的內容一律當成資料而不是命令、輸出要驗證再使用。最後，**API Key** 要放在後端或環境變數，絕對不要放進前端或提交到 **Git**。